

Parallel Travelling Salesman Problem Solver

Island-Model Genetic Algorithm with MPI

This report describes a parallel solver for the Travelling Salesman Problem (TSP). The implementation uses an island-model Genetic Algorithm (GA) written in C++ and parallelized with MPI. Each MPI process is an independent island that evolves a population of tours. At fixed synchronization intervals, all islands cooperate by finding the global best tour with MPI_Allreduce and broadcasting the owner tour with MPI_Bcast. Other islands do not clone the whole tour; instead, they splice random contiguous segments of the global best into their own population through order crossover. This keeps the search diverse while still spreading useful sub-routes. The repository also contains Python tools for visualization, validation, and plot generation. The Python code does not implement the TSP algorithm; it only drives the C++ executable or reads output files generated by the solver. The experimental results cover correctness validation, input-size selection, granularity/load balance, and speedup with and without communication time.

Project Overview

Problem Statement

Given a set of N cities with two-dimensional coordinates, the Travelling Salesman Problem asks for a closed tour that visits every city exactly once and returns to the starting city while minimizing total Euclidean distance. For a tour $T = (t_0, t_1, \dots, t_{N-1})$, the objective value is:

$$L(T) = \sum_{i=0}^{N-1} d(t_i, t_{(i+1) \bmod N})$$

where $d(a,b)$ is the Euclidean distance between city a and city b . TSP is NP-hard, so the project uses a heuristic search method instead of exact enumeration for large instances.

Repository Structure

The project is organized as follows: sequential baseline, and unit tests.

orchestration tools.

cluster launch scripts.

Main Source Files

The full solver is in C++:

evaluates tours, creates random tours, performs tournament selection, order crossover (OX), mutation, and one generation of evolution.

unit tests for genetic operators and local search.

Sequential Genetic Algorithm

Representation

Each individual is a permutation of city indices. For example, for $N=5$, a tour may be:

The tour is closed implicitly, so the last city connects back to the first city.

This representation guarantees that a valid individual can be evaluated directly as a TSP route.

Fitness Evaluation

The solver precomputes a flat $N \times N$ Euclidean distance matrix. The fitness value is the total tour length. Smaller values are better.

Selection, Crossover, and Mutation

The GA uses:
and fills the remaining positions using the order of the second parent.

Local Search

The optional memetic extension applies local search to the best individual every segment.

Parallelization Design

Parallelism Level

The implementation uses coarse-grained task-level parallelism. Each MPI process owns a complete GA population and performs an independent stochastic search.

The city data and distance matrix are replicated on every process.

The design also has an exploratory-search characteristic: different islands explore different regions of the search space due to different random seeds.

Therefore, the decomposition is best described as a hybrid of task decomposition

and exploratory decomposition.

Decomposition Technique

The selected decomposition is exploratory decomposition with replicated input data. Each island receives the same TSP instance but evolves a different population. This is suitable for a heuristic optimization problem because the main work is search, not deterministic matrix processing.

The alternatives were less suitable:
the global order of all cities.
search, not for the selected GA.
not the focus of this project.

Mapping Technique

The mapping is a one-dimensional process-to-island assignment:

Each rank stores:

For the cluster experiment, the hostfile uses a sequential round-robin list of hosts. This spreads ranks across four machines and supports up to 48 ranks, with 12 ranks per machine as the common physical-core limit.

Communication Strategy

Communication is periodic and collective. Every -sync generations:

length and the owner rank.

global best with local individuals.

This is a blocking collective communication strategy.

The logical topology is

global collective communication rather than a ring or master-slave topology.

There is no permanent master rank during evolution; all ranks participate

symmetrically. Rank 0 is used only for final reporting and file output.

Migration Policy

A naive island GA can broadcast the full best individual and clone it into every island. That is fast but dangerous: diversity collapses and all islands may converge to the same local optimum.

This implementation uses partial global-best migration: random local mate.

Only sub-routes are shared. Random cut points and local mates keep the resulting population different on each island.

Load Balancing Considerations

In the default mode, every rank uses the same population size, so compute work is approximately balanced. This is appropriate for machines with similar effective capacity or for a controlled experiment where ranks per machine are limited by the slowest machine.

The solver also contains an optional `-auto-balance` mode. It performs a small local benchmark on each process, gathers the measured times, and assigns a larger population to faster ranks. The allocation is clamped between 0.5x and 2x of the original population to avoid unstable decisions from noisy measurements.

Early Stopping

If `-patience` is enabled, the solver stops when the global best does not improve for a fixed number of generations. Because the same global best value is known to all ranks after each synchronization, every rank can make the same stop decision without an extra control message.

Parallel Algorithm Pseudocode

Pseudocode / command listing omitted in preview PDF.

Implementation and Instrumentation

Command-Line Interface

The main solver accepts the following key arguments: best per synchronization.

Timing Metrics

The solver separates communication time from total elapsed time:

These metrics are written to the `-stats` CSV file and used by

Cluster Environment

The cluster setup uses four machines and up to 48 MPI ranks. OpenMPI 5.0.9 is source-built and pinned at `/opt/openmpi-5.0.9` on all nodes to avoid runtime incompatibilities. The launch script uses: heterogeneous machines.

Correctness Validation

Permutation Validity

The solver represents a route as a permutation of city IDs. The unit tests check that crossover and mutation preserve valid

permutations. The validation script Small-Instance Exact Check

For N 10, the validation script can brute-force all tours with city 0 fixed as the first city. This gives the true optimum and allows the GA result to be checked against the exact solution. The repository includes validation logs for small and larger instances in results/.

Parallel Communication Check

The per-rank history files show that migration changes the behavior of non-owner islands after synchronization points. In one cluster run, rank 2 held the best tour before synchronization at generation 20. After the synchronization, weaker islands moved toward that better solution, but did not become identical. This matches the intended partial-migration behavior: useful route segments spread, while population diversity is preserved.

Experimental Results

Experiment 1: Input Size Selection

The requirement asks for an input-size experiment using the number of processes equal to the number of physical CPU cores. The cluster configuration uses up to 48 ranks. The runtime is measured in two cases: total time including communication, and estimated compute time excluding communication.

The first sweep used a fixed 400-generation run to understand the growth trend.

It showed that larger instances increase compute work and communication volume, but the run was still too short for the required 2-3 minute target.

N | Total time (s) | Communication (s) | Compute only (s)

100 | 1.2427 | 1.1512 | 0.0915

200 | 1.0223 | 0.8770 | 0.1453

400 | 1.7410 | 1.4027 | 0.3383

800 | 2.9824 | 1.9272 | 1.0552

1200 | 4.2491 | 2.4806 | 1.7685

2400 | 8.1918 | 3.8092 | 4.3826

[Figure: ../results/exp_size.png]

To satisfy the 2-3 minute requirement, the final selection fixed

N | Total time (s) | Communication (s) | Compute only

(s)

1200 | 54.54 | 25.90 | 28.65

1800 | 86.99 | 34.33 | 52.66

2400 | 154.27 | 64.05 | 90.22

The selected input size is therefore:

The official repeated run stored in

results/nstar_run.log reports a

makespan of 145.97 seconds, which lies inside the

required 120-180

second interval. Other repeated measurements at the

same configuration were

159.14s and 154.27s, also inside the target range.

Experiment 2: Granularity and Load Balance

For the granularity experiment, the program is run with

the selected input size

$N^*=2400$, gens=10500, and 48 ranks. Each rank is one

island, so the

granularity is one GA population per process. The

required stacked bar chart

shows compute time and communication time in the same

column for each process.

[Figure: ../results/exp_gran.png]

The measured idle skew is about 1.8\

threshold. Therefore, the system is considered load-

balanced for this run. The

main difference between ranks is not idle waiting but

the split between compute

and communication time, which is expected when ranks

synchronize through

blocking MPI collectives.

Experiment 3: Speedup

The speedup experiment uses a fixed total workload and

varies the number of

processes. Following the requirement, this experiment

uses $2N^* = 4800$ cities

with gens=400, sync=20, and a fixed total population of

480

individuals split over the active ranks. The measured

values are:

Procs | Total (s) | Comm (s) | Compute (s) | Speedup |

No-comm speedup | Eff.

1 | 16.3636 | 0.0000 | 16.3636 | 1.0000 |

1.0000

2 | 9.6910 | 3.6071 | 6.0840 | 1.6885 | 2.6896 | 0.8443

4 | 4.7995 | 2.1130 | 2.6865 | 3.4094 | 6.0910 | 0.8524

8 | 3.3492 | 1.9690 | 1.3802 | 4.8858 | 11.8559 |

0.6107

16 | 2.8577 | 2.0346 | 0.8231 | 5.7261 | 19.8806 |

0.3579

32 | 3.6413 | 2.6965 | 0.9448 | 4.4939 | 17.3200 |

0.1404

48 | 4.0388 | 2.8897 | 1.1491 | 4.0516 | 14.2398 |

0.0844

[Figure: ../results/exp_speedup.png]

The speedup improves up to 16 processes and then decreases. This means the parallel computation scales initially, but the communication and synchronization overhead becomes dominant at high process counts. The no-communication speedup is much higher, showing that the GA computation itself is parallelizable. The limiting factor is the cost of repeated collective communication on the real cluster network.

Discussion

Why the Island Model Fits TSP

The island model is a natural fit for TSP because the problem is a global combinatorial search. Multiple populations can explore different areas of the solution space without requiring communication at every generation. This reduces the cost compared with fine-grained parallel GA designs.

Communication Trade-Off

The -sync parameter controls the trade-off between cooperation and overhead. A small interval spreads good information quickly but increases communication cost and can reduce diversity. A large interval reduces overhead but makes the islands behave more like independent runs. The no-sharing mode

Granularity

The chosen granularity is coarse: one full GA population per process. This reduces communication frequency and makes the program easier to run on a heterogeneous cluster. The drawback is that very small TSP instances do not have enough computation per process, so communication dominates.

Limitations

The current implementation is a heuristic solver. It does not guarantee the global optimum for large TSP instances. The exact check is only feasible for

small N . The selected $N^*=2400$ configuration satisfies the 2-3 minute runtime target, but the result is still affected by normal run-to-run variation on a heterogeneous laptop cluster.

Conclusion

This project implements a complete MPI-based parallel TSP solver using an island-model Genetic Algorithm. The parallelism is coarse-grained and task/exploratory in nature. Each MPI rank owns an island and communicates only periodically through blocking MPI collectives. The mapping is one-dimensional: rank to island. The communication topology is global collective communication using `MPI_Allreduce` and `MPI_Bcast`. Correctness is checked through permutation tests, independent route-length validation, and brute-force validation for small instances. The experimental results show balanced work distribution and meaningful speedup up to a moderate number of processes. At high process counts, communication overhead dominates, which is visible in the difference between total speedup and no-communication speedup.

Appendix: Reproducible Commands

Pseudocode / command listing omitted in preview PDF.